# Práctica 5: Integración de un sistema embebido (aplicación IoT)

Fundamentos de Sistemas Embebidos  Semestre 2027-1

## 1 Objetivo

Integrar una aplicación IoT utilizando una Raspberry Pi como plataforma embebida, mediante la comunicación entre diferentes componentes de software a través del protocolo MQTT.

El sistema deberá adquirir una medición de temperatura, publicarla mediante MQTT y permitir el monitoreo y control de un actuador desde una interfaz web.

## 2 Introducción

En los sistemas embebidos modernos es común que diferentes componentes de software deban intercambiar información. Por ejemplo, un programa puede adquirir datos de sensores mientras otro procesa la información o controla actuadores.

MQTT es un protocolo ligero de comunicación basado en el modelo publicación/suscripción. Los clientes intercambian mensajes mediante un intermediario denominado *broker*, utilizando canales identificados como *tópicos*.

Esta arquitectura permite separar las responsabilidades de adquisición, procesamiento y control, facilitando la integración de diferentes componentes.

En esta práctica se utilizará MQTT para conectar un sensor de temperatura, un actuador y una interfaz web dentro de una aplicación embebida.

## 3 Material requerido

- Raspberry Pi con sistema operativo configurado.
- Sensor de temperatura utilizado en prácticas anteriores.
- Ventilador de corriente directa pequeño.
- Controlador de corriente L293D o equivalente.
- Fuente externa adecuada para el ventilador.
- Protoboard y cables de conexión.
- Computadora o teléfono para acceder a la interfaz.
- LED y resistencia (alternativa al ventilador).

**Nota:** Para simplificar el desarrollo, el ventilador podrá sustituirse por un LED con resistencia. La lógica de control será equivalente.

## 4 Arquitectura del sistema

La aplicación estará compuesta por tres elementos principales:

1. **Publicador de temperatura:** adquiere y publica las mediciones del sensor.
2. **Controlador del actuador:** recibe comandos MQTT y modifica el estado de una salida GPIO.
3. **Interfaz web:** muestra la temperatura y permite enviar comandos de encendido y apagado.

Todos los componentes se comunicarán a través de un broker MQTT ejecutado en la Raspberry Pi.

### 4.1 Tópicos MQTT

Se utilizarán los siguientes tópicos:

| Tópico | Descripción | Mensaje |
|---|---|---|
| `/iot/temp` | Temperatura medida | Decimal |
| `/iot/fan/cmd` | Comando del actuador | true / false |
| `/iot/fan/status` | Estado del actuador | true / false |

Los valores `true` y `false` se transmitirán como cadenas de texto, representando encendido y apagado, respectivamente.

### 4.2 Consideraciones eléctricas

Los GPIO de la Raspberry Pi trabajan con niveles lógicos de 3.3 V y no deben utilizarse para alimentar directamente motores o ventiladores.

Para utilizar el ventilador se deberá implementar una etapa de potencia mediante el controlador L293D o un circuito equivalente, utilizando una fuente de alimentación externa adecuada.

El GPIO únicamente proporcionará la señal de control. Verificar que las tierras de la Raspberry Pi y de la etapa de potencia tengan una referencia común, cuando el circuito lo requiera.

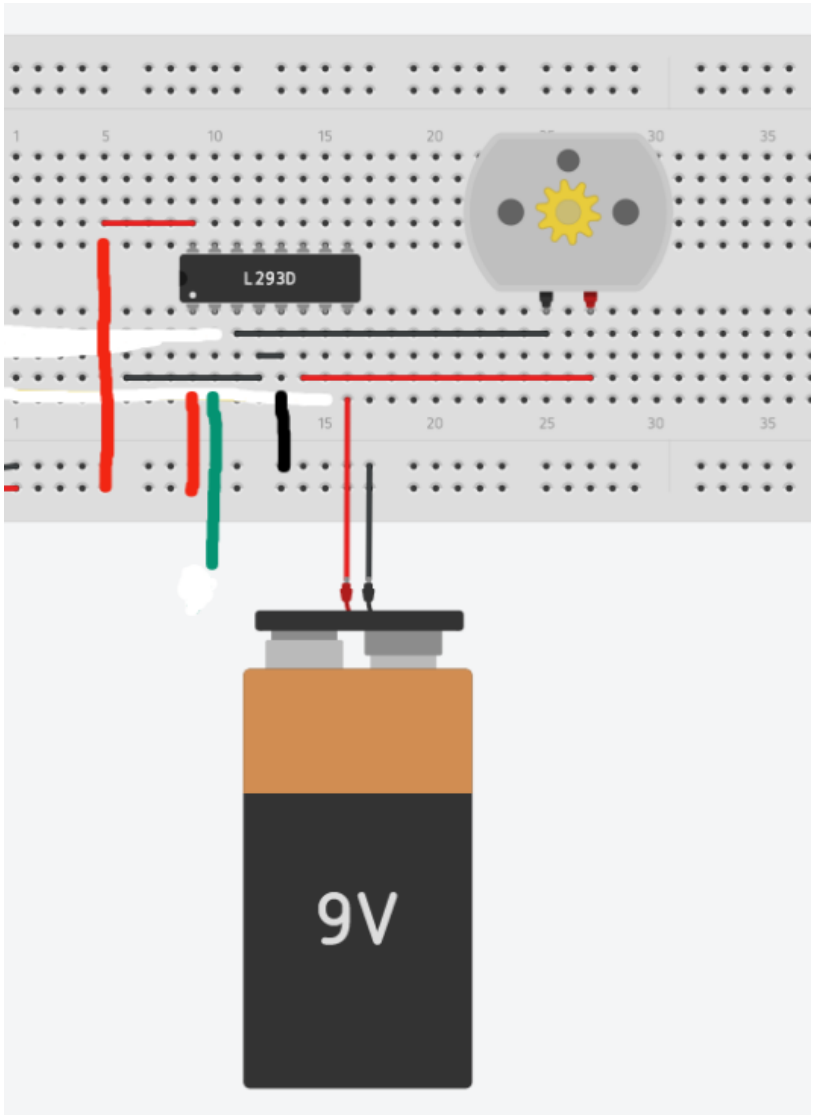



Figura 1: Conexión del controlador y el ventilador.

## 5 Desarrollo

### 5.1 Preparación y prueba de MQTT

Instalar el broker Mosquitto y las herramientas de prueba en la Raspberry Pi:

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

Iniciar y verificar el servicio:

```
sudo systemctl enable --now mosquitto
sudo systemctl status mosquitto
```

Para comprobar su funcionamiento, abrir una terminal y suscribirse al tópico:

```
mosquitto_sub -h localhost -t /iot/temp
```

En otra terminal, publicar una medición de prueba:

```
mosquitto_pub -h localhost \
    -t /iot/temp -m "27.5"
```

Verificar que el mensaje aparezca en la terminal del suscriptor.

### 5.2 Publicación de temperatura

Desarrollar un programa que obtenga periódicamente la temperatura del sensor y publique el resultado en el tópico `/iot/temp`.

El programa deberá:

- Obtener la temperatura mediante el controlador o biblioteca utilizada en prácticas anteriores.
- Publicar una medición cada segundo.
- Enviar únicamente el valor numérico.
- Mostrar las lecturas en la terminal.

Como referencia, se muestra la estructura básica de un publicador MQTT en Python:

```
import time
import paho.mqtt.client as mqtt

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)
client.connect("localhost", 1883, 60)
client.loop_start()

try:
    while True:
        temperatura = 28.5  # Sustituir por sensor
        client.publish("/iot/temp", str(temperatura))
        time.sleep(1)
finally:
    client.loop_stop()
    client.disconnect()
```

Instalar la biblioteca cuando sea necesario en el entorno Python utilizado:

```
python3 -m pip install paho-mqtt
```

**Nota:** En Raspberry Pi OS puede ser necesario utilizar un entorno virtual de Python para instalar bibliotecas mediante `pip`.

### 5.3 Control del actuador

Desarrollar un programa que reciba comandos MQTT desde el tópico:

`/iot/fan/cmd`

El programa deberá:

- Encender el actuador al recibir `true`.
- Apagarlo al recibir `false`.
- Ignorar comandos no reconocidos.
- Publicar su estado en `/iot/fan/status`.

El siguiente ejemplo muestra la estructura de recepción de mensajes:

```
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags,
               reason_code, properties):
    if not reason_code.is_failure:
        client.subscribe("/iot/fan/cmd")

def on_message(client, userdata, msg):
    comando = msg.payload.decode().strip()

    if comando == "true":
        print("Encender actuador")
        # Activar GPIO
    elif comando == "false":
        print("Apagar actuador")
        # Desactivar GPIO
    else:
        return

    client.publish("/iot/fan/status", comando)

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883, 60)
client.loop_forever()
```

Completar el programa con las instrucciones necesarias para controlar el GPIO seleccionado.

Verificar su funcionamiento enviando comandos desde otra terminal:

```
mosquitto_pub -h localhost \
    -t /iot/fan/cmd -m "true"

mosquitto_pub -h localhost \
    -t /iot/fan/cmd -m "false"
```

### 5.4 Interfaz web de monitoreo y control

Implementar una interfaz web sencilla que permita interactuar con el sistema.

La interfaz deberá incluir:

- Visualización de la temperatura actual.
- Indicador del estado del actuador.
- Botón para encender el actuador.
- Botón para apagar el actuador.

La interfaz deberá comunicarse con el broker MQTT para consultar los tópicos de temperatura y estado, así como publicar los comandos correspondientes.

Se permite utilizar herramientas o bibliotecas que faciliten la implementación de la interfaz.

**Nota:** Si la interfaz se ejecuta en un navegador y utiliza MQTT directamente, será necesario habilitar una conexión WebSocket en el broker. Como alternativa, podrá utilizarse un servidor web que actúe como intermediario entre el navegador y MQTT.

## 6 Prueba de integración

Ejecutar simultáneamente los componentes desarrollados y verificar lo siguiente:

1. El sensor publica las mediciones de temperatura mediante MQTT.
2. La interfaz web muestra la temperatura recibida.
3. Los botones de la interfaz permiten encender y apagar el actuador.
4. El estado del actuador se publica correctamente en MQTT.
5. La interfaz refleja el estado real reportado por el controlador.

Durante la demostración, verificar que los programas se comuniquen mediante MQTT y no mediante variables compartidas o llamadas directas entre ellos.

## 7 Actividad complementaria (opcional)

Implementar un control automático ON/OFF que active el ventilador cuando la temperatura alcance un umbral definido.

Incorporar histéresis para evitar cambios continuos de estado.

Por ejemplo:

- Encender cuando $T \geq 30\,°C$.
- Apagar cuando $T \leq 28\,°C$.

- Conservar el estado entre ambos umbrales.

El control automático deberá publicar sus comandos mediante el mismo tópico utilizado por la interfaz web.

## 8 Preguntas de análisis

1. ¿Qué función cumple el broker MQTT dentro del sistema desarrollado?
2. ¿Por qué es conveniente separar la adquisición de temperatura, el control del actuador y la interfaz web?
3. ¿Cuál es la diferencia entre el tópico de comando y el tópico de estado del actuador?
4. ¿Qué sucedería si el broker MQTT dejara de funcionar durante la ejecución?

## 9 Entregables

Presentar durante la sesión:

- Demostración del sistema integrado.
- Código fuente de los componentes desarrollados.
- Evidencia de la comunicación MQTT.
- Respuestas a las preguntas de análisis.